BRIGHTLOOP

Functional, Integration & Security Test Report

Fresh audit of the updated Subscription Management System

Audited snapshot	df4ee28a0ec4fe1808d751c803da3452f9869283
Audit date	31 October 2026
Test stack	Playwright / Chromium, Express HTTP, SQLite, PostgreSQL 17, controlled SMTP
Fresh checks	65 new named checks: 60 pass, 5 fail
Revised assessment	85 / 100
Release recommendation	HOLD — critical reset-token exposure

Covers the billable month, customer portal, roles, validation, database parity, mail delivery, local performance, mobile layout and accessibility. Evidence files and limits are listed in Appendix C.

Contents

1. The short version
2. Scope, version and method
3. A complete billing month, exercised through the UI
4. Roles, portal and day-to-day operations
5. Integration and resilience findings
6. Performance and original-layout checks
7. Confirmed defect register
8. Requirement-by-requirement evaluation
9. Revised score and release judgement
10. What remains untested or intentionally outside scope
Appendix A. New test register
Appendix B. Existing regression inventory
Appendix C. Evidence and reproducibility

Functional, Integration & Security Test Report

Fresh audit of the updated Subscription Management System

31 October 2026 · Node 24 · Playwright / Chromium · SQLite and PostgreSQL 17

Revised assessment: 85 / 100

Release recommendation: HOLD — one critical security finding remains open.

This report records what the application actually did during a fresh automated review. It is not a rewording of the earlier 93-point evaluation. The ordinary billing journey worked, but a new cross-role password-reset test exposed an administrator-account takeover path. That result takes precedence over the earlier score and over a green test dashboard.

Prepared from executed browser journeys, HTTP integration tests, controlled SMTP delivery, database assertions and source inspection. No customer interviews, human usability panel, external mailbox delivery or independent penetration-test certification are implied.

1. The short version

The most encouraging part of the review was the complete commercial journey. I created a product with a separate cost price, added the Odoo brand adjustment, configured a plan, prepared a reusable offer, converted it, activated a second contract and generated its invoice. The invoice came to exactly $1,306.00. A controlled SMTP receiver accepted its PDF attachment. A $100 bank payment left the invoice open; a $1,206 UPI payment cleared it. The customer could then schedule a seat increase and add-on without rewriting the paid invoice.

The main concern is not a cosmetic edge case. A staff account is normally blocked from adding staff or changing administrator-controlled catalog data. However, the same staff account can read notification bodies. Those bodies include administrator password-reset URLs. In an isolated test, that was enough to set a new administrator password and sign in as ADMIN. The direct permission checks passed; the indirect path defeated them.

Four other checks failed: an advanced catalog selector stopped at 100 records, the overview had insufficient text contrast, Tab escaped the advanced-editor modal, and the settings screen described local logging even while the application was sending SMTP mail.

The useful conclusion is therefore mixed: **the business implementation is substantially stronger than the original prototype, but the current build should not be released with real accounts until the reset-token exposure is fixed.**

1.1 Results at a glance

Workstream	Fresh result	How to read it
Existing SQLite integration, regression and upgrade suite	39 passed; 0 failed	Automated test cases, not a claim that every possible behavior is covered
Existing PostgreSQL 17 suites	38 passed; 0 failed	15 core + 23 hardening tests; the SQLite legacy-upgrade fixture is not duplicated here
Existing demo-browser suite	10 passed	Browser-only demonstration behavior; not proof of backend authorization
Existing API-connected browser suite	10 passed	Real REST calls and database writes, including contract editing and customer isolation
New exploratory browser/API journey	35 passed; 5 failed	40 named checks, including the portal count reconciliation, accessibility, catalog scale and transport-label checks
New API edge-case matrix	25 passed	Fresh validation, state, frequency, permission and retry checks
New cross-role security probe	1 failed security expectation	A staff account successfully reached an administrator account through a leaked reset URL
Combined NEW named checks	**60 passed; 5 failed; 65 total**	Browser 40, API matrix 25; test families overlap and this is not a product-completeness percentage
Lint, TypeScript and production build	Passed	The source compiles and satisfies the configured static checks
Backend coverage	89.04% lines; 77.70% branches; 83.24% functions	Loaded non-test backend modules; not frontend, infrastructure or security coverage
Dependency audit	0 known findings at the recorded run	Does not detect the first-party authorization defect found here
Native production / backup smoke	Passed	Static/API/auth headers and SQLite backup integrity; not a full deployment rehearsal
Original-style comparison	Passed	Two CSS hashes and eight measured regions at 1440px and 390px; not pixel-perfect proof of every screen

The 38 PostgreSQL executions repeat the core behaviors against another database driver. They should not be described as 38 entirely new business scenarios. Likewise, the exploratory journey deliberately revisits important paths from the existing browser suites.

2. Scope, version and method

2.1 What was reviewed

The reference requirement was the supplied four-page Subscription Management System problem statement: accounts and roles; products, variants and plans; quotations and lifecycle; recurring billing; taxes and discounts; payments; customer self-service; and finance reporting with thousands of subscriptions.

The repository snapshot was `df4ee28a0ec4fe1808d751c803da3452f9869283`. The implementation snapshot beneath the release archive is `74d1f94`. The audit did not patch application code, commit changes or push anything to GitHub. Source hashes and a clean working-tree record are in `logs/provenance.json`.

All destructive operations were confined to freshly seeded, isolated test databases. No live customer, production mailbox or real payment rail was used. Payment methods such as bank, card and UPI are accounting labels here: the software records receipts; it does not initiate transfers.

2.2 Test layers

- **Browser execution:** Playwright drove the real production-built React UI in headless Chromium 156.0.8078.4. The custom journey used 1440 × 1000 desktop and 390 × 844 mobile viewports.
- **Real API and persistence:** the browser called Express over HTTP. SQLite assertions checked the resulting states, prices, revisions and invoice balances. No mocked API responses were substituted for the main business journey.
- **API-only negatives:** Supertest exercised the same route implementation in isolated processes. This is integration testing, not browser interaction, and is labelled accordingly.
- **Database parity:** the repository's core and hardening suites ran against PostgreSQL 17 using isolated schemas. The schemas were dropped afterwards.
- **Email:** a local SMTP server accepted actual SMTP sessions and MIME messages. Tests inspected a PDF attachment, a reset link, a failed delivery and one successful retry under two competing workers. TLS and external-provider authentication were deliberately not part of this loopback test.
- **Accessibility:** axe-core checked the admin overview against selected WCAG 2 A/AA and 2.1 AA rules. A separate keyboard probe checked modal focus containment. Neither is a complete accessibility certification.
- **Load:** a mixed ledger of 10,000 subscriptions, 60,000 invoices and 30,000 payments was used for HTTP reads and real browser navigation.

2.3 What “simulated” means in this report

The company, customers, money and email recipients were synthetic. The browser events, server routes, SQL transactions, PDF bytes and SMTP exchanges were real. One automatic-expiry test moved a fixture's expiry to today's boundary; it did not wait for the daily cron clock. This distinction matters: a successful direct billing call does not prove that a production scheduler will run unattended after a restart.

The first attempt to run multiple frontend builds at once was killed under the sandbox's approximately 2 GB memory limit. Those results were not counted as application failures or performance samples. After stopping the unrelated preview/database processes, the production acceptance build ran sequentially and passed. The final acceptance harness explicitly set `NODE_ENV=production` for the frontend build; application logic was not altered.

An early visual-comparison attempt also timed out while the parallel builds were active. The subsequent isolated comparison passed. Both attempts are retained in the evidence so that a failed setup run is not quietly presented as a successful test.

2.4 Evidence conventions

PASS means that the stated expectation was observed. FAIL means that a valid expectation was violated. Source-reviewed means that the code was inspected but the behavior was not fully exercised in the stated environment. Not run means no claim is made.

Screenshots are Playwright captures, not generated mock-ups. The report uses viewport crops and focused dialog crops for readability; full PNGs are supplied separately. The reset-token field is deliberately masked. A screenshot illustrates state at a point in time; status codes, database assertions and the JSON register provide the stronger evidence for correctness.

3. A complete billing month, exercised through the UI

3.1 Getting into the workspace

A deliberately wrong administrator password produced a 401 and an inline error instead of opening the workspace. The correct seeded credentials then loaded the API-connected dashboard. The test verified the server-reported ADMIN role rather than relying on a role label in the interface.

The overview CSV was downloaded and inspected for its financial summary headings. “Last month” selected the previous calendar month, September 1–30, and “This month” restored October. These are useful everyday checks: the dashboard is only meaningful if its date controls and exports agree about what is being shown.

Figure 1. Actual API-connected sign-in screen. Wrong credentials were rejected before the successful administrator login.

Figure 2. Administrator overview on the isolated seeded database. This is not the high-volume fixture or the browser-only demo.

3.2 Product, variant and plan setup

Through Advanced setup, the audit created **Audit Ledger**, a Software product with a $100 sales price, a $20 cost price and a note. A database read confirmed that sale and cost were stored independently. The Odoo option was then created in the Brand group with a +$560 adjustment.

The plan, **Audit Monthly**, used a $100 plan price, monthly frequency and a two-seat minimum. Availability dates and the pause, renewal, early-close and auto-close flags were persisted. The separate API matrix accepted daily, weekly, monthly and yearly frequencies, rejected a reversed availability window and rejected an unavailable plan at subscription creation.

The effective unit price for the main contract was therefore $660: $100 plan price plus $560 for the selected brand. The UI did not get to supply an arbitrary authoritative unit price; the server rebuilt it from the catalog.

Figure 3. Product editor used in the audit. Sale price, cost price, notes and type were written to the real database.

Figure 4. Billing-plan setup with minimum seats, availability and per-plan lifecycle permissions.

3.3 Tax and discount setup

The UI rejected a percentage of 101 before saving. A valid fixed tax of $2 per unit was then scoped to Audit Ledger. A fixed $20 product discount was configured with a minimum amount of $100, a minimum quantity of two, start/end dates and a maximum of five uses.

The existing seed supplied a 10% sales tax and a 10% subscription-wide welcome discount. The expired and exhausted seed discounts were not supposed to qualify. That combination made the resulting invoice a useful arithmetic check rather than a trivial “total equals price” test.

The existing regression suites additionally covered capped-discount confirmation races, expired and exhausted rules, historical snapshots and both database drivers. The custom UI journey confirmed creation and actual application of the new rules; it did not independently enumerate every possible rule combination.

Figure 5. Product-scoped fixed discount with purchase/quantity thresholds, date limits and a usage cap.

3.4 Quotation and conversion

The browser created **Audit onboarding**, a reusable template with the audit plan, a two-seat product line and 14-day validity. A quotation was created for the seeded customer and converted using the visible Convert action. A second conversion attempt returned 409, which is the right outcome for a single-use offer.

The main billing contract was created separately with the Odoo variant. This avoids confusing the plain template's price with the brand-adjusted contract used for the arithmetic proof. Both were real records, but only the brand-adjusted contract was taken through the complete payment journey.

The pre-existing price-freeze regression edited the catalog after making an offer and checked that conversion retained the agreed quote price. Expired-quotation rejection and conversion atomicity were also rerun in the existing suites.

Figure 6. Real quotation, customer, expiry and conversion action. A repeated conversion was rejected.

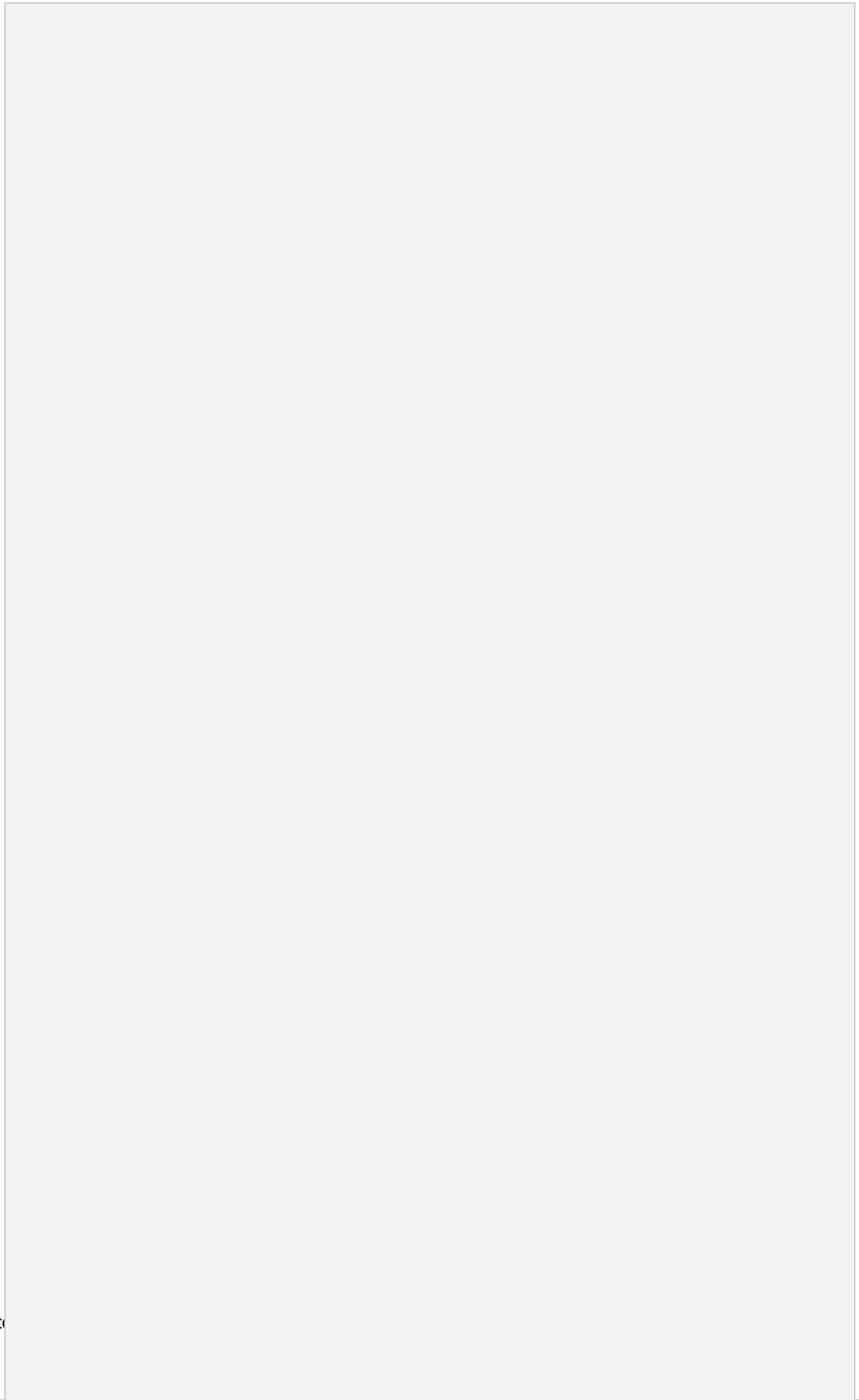


Figure 7. Main test contract: two seats, Odoo brand adjustment, dates and Net 14 terms.

3.5 Subscription lifecycle

The browser drove the main contract through Draft → Quotation → Confirmed → Active. It then paused and resumed it. Each step was checked against the database, not just the badge in the drawer.

The API matrix separately rejected activation directly from Draft, allowed cancellation of a Draft, blocked actions disabled by plan policy, extended an active contract by one monthly period and rejected a stale amendment revision. A clock-aged daily fixture auto-closed at its expiry boundary. Existing tests covered accrued debt across pause/resume and early closure, completed pause intervals, month-end recovery and leap-year anniversaries.

The implementation uses full anchored periods without proration. A pause does not automatically extend expiry. Those are explicit product policies, not outcomes a customer should have to infer from the next bill. They should be explained in commercial terms before a real launch.

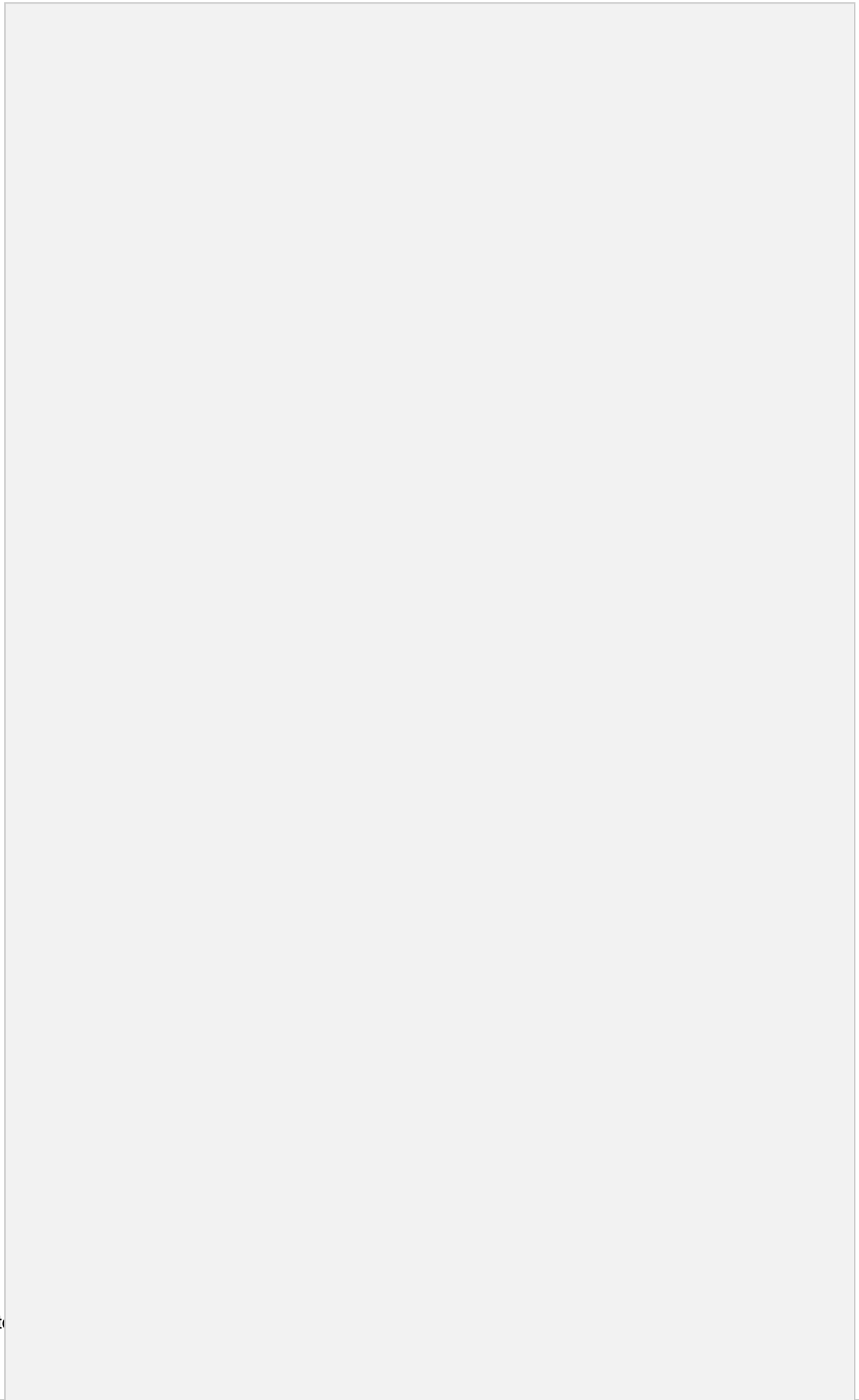


Figure 8. Contract drawer after the successful lifecycle sequence. The server state and displayed status were checked together.

3.6 Billing, arithmetic and invoice state

The visible Run billing control invoked the real backend. It created an invoice draft for the brand-adjusted contract. Here is the independently checked calculation:

Step	Calculation	Amount
Base service	2 × ($100 plan + $560 Odoo adjustment)	$1,320.00
Standard sales tax	10% × $1,320	$132.00
Audit fixed tax	2 units × $2	$4.00
Product discount	Fixed amount, threshold satisfied	−$20.00
Welcome discount	10% × ($1,320 − $20)	−$130.00
Invoice total	$1,320 + $136 − $150	**$1,306.00**

This is the application's documented ordering: tax is evaluated before discount, and product discounts reduce the basis for subsequent subscription discounts. It is not a claim of tax-jurisdiction compliance. The invoice retained the same total on confirmation. Running billing again did not add another invoice for the same subscription period.

Figure 9. Browser billing result from the real service. This was not a local demo-only insertion.

Figure 10. Generated invoice draft: subtotal $1,320, tax $136, discount $150 and total $1,306. The report crop focuses on the amounts.

3.7 Sending, printing and paying

The confirmed invoice was sent through a real SMTP session to a controlled receiver. The captured MIME message contained an application/pdf attachment and the invoice number. The browser also downloaded a PDF; its first bytes were %PDF, not an HTML error page with a .pdf extension.

A $100 bank payment, with date and note, left the invoice Confirmed with a $1,206 balance. Recording the exact $1,206 remainder using UPI changed it to Paid with $1,306 received. These are receipt records, not processor settlements.

The API matrix rejected zero, sub-cent, future-dated and excessive payments. It checked cancellation, a stable payment idempotency key and conflict on changing a payload under that key. The existing suites supplied the stronger concurrent and PostgreSQL coverage. Refunds, chargebacks, bank-feed reconciliation and live payment-gateway callbacks were not implemented or simulated as if they existed.

Figure 11. Payment form used for the first receipt: invoice, amount, method, date and note.

Figure 12. Same invoice after the partial and final receipts. The balance is zero and the status is Paid.

3.8 Amendments after activation

The administrator changed the contract to Net 30 and extended its expiry. A database assertion confirmed the new terms. The already issued $1,306 invoice did not change.

A separate customer session then changed the requested quantity to three and added Analytics Plus. The customer editor omitted expiry and payment terms. The database kept the current two-seat lines intact and saved two pending lines with a future effective date. This is a meaningful improvement over silently rewriting an active contract's history.

Figure 13. Staff amendment form. Commercial terms are available here but not in the customer editor.

Figure 14. Customer-owned seat/add-on proposal. Current service remained unchanged until the future billing boundary.

4. Roles, portal and day-to-day operations

4.1 Legitimate access boundaries

The customer logged in with an actual CUSTOMER session, not the demo role switcher. Staff-only navigation was absent. Foreign subscription detail, a foreign invoice PDF, a foreign amendment and another customer's list filter were rejected with 403. The customer-safe catalog omitted cost price and notes.

The staff session could work with operational pages but could not create products or staff through direct API calls. Only the administrator created the audit operator through the team dialog. Logout revoked an existing access token. Refresh tokens rotated and a replayed refresh token was rejected.

These direct boundaries are real strengths, but they do not cancel the reset-link defect in section 7. A role model has to survive the combination of endpoints, not just an isolated permission middleware check.

Figure 15. Actual customer portal. Navigation and data are scoped by the authenticated session.

4.2 Search, exports and operational visibility

Global server search found the new contract. The existing API-browser suite found a customer beyond the first 100 rows in the searchable picker. The high-volume journey repeatedly visited subscription, invoice, payment, customer, quotation, report, audit and configuration pages without leaving old record tables on the overview.

The audit trail recorded the tested contract's mutations. Invoice and subscription lists used server-side filters and pagination. The API matrix checked bounded report reads, a page-size cap of 100, reversed date-range rejection, billing history, upcoming renewals and reminder-key deduplication.

CSV exports and PDF downloads are useful features, but their scope must be understood. A paginated table's “Export page” is a page export, not automatically a full-ledger extraction. The financial CSV checked in this journey was the overview summary. Spreadsheet-formula escaping exists in the download helper; this review did not execute downloaded files inside Microsoft Excel or LibreOffice.

Figure 16. Global search finding the newly created contract through the authenticated server.



Figure 17. Audit log populated by the real workflow. Full original screenshots are supplied for zooming.

4.3 Settings and device-local behavior

Changing the workspace display name persisted to localStorage. Currency was visibly fixed to USD and billing timezone to UTC. This is a device-local preference screen, not a server-wide organization-settings module.

One label was wrong in the tested configuration: Email delivery still said “Console + outbox.log” although the same process had successfully delivered through SMTP. The scheduler label is also static rather than a live health indication. Operators should not rely on either label as proof of a running worker or a healthy provider.

Figure 18. Settings screen during the SMTP-backed audit. Its static delivery description is finding F-05.

4.4 Mobile and accessibility

At 390px, the customer overview and invoice list fit without horizontal page overflow. The navigation drawer opened and the portal remained usable in the automated journey. That is a responsive-layout result; headless Chromium with a narrow viewport is not a physical Android-device test.

The overview failed axe-core's color-contrast rule on **136 affected nodes**. That is one violated rule with many instances, not 136 independently discovered security defects. A keyboard probe also moved focus out of the open advanced-editor modal and onto the page body. The app therefore needs accessibility work even though its original visual identity was preserved.

The unchanged CSS helps explain why readability issues survived the previous “keep it visually identical” constraint. The appropriate fix is a restrained contrast and focus treatment, not a redesign. Any resulting style change should be agreed explicitly against that constraint and rechecked visually.

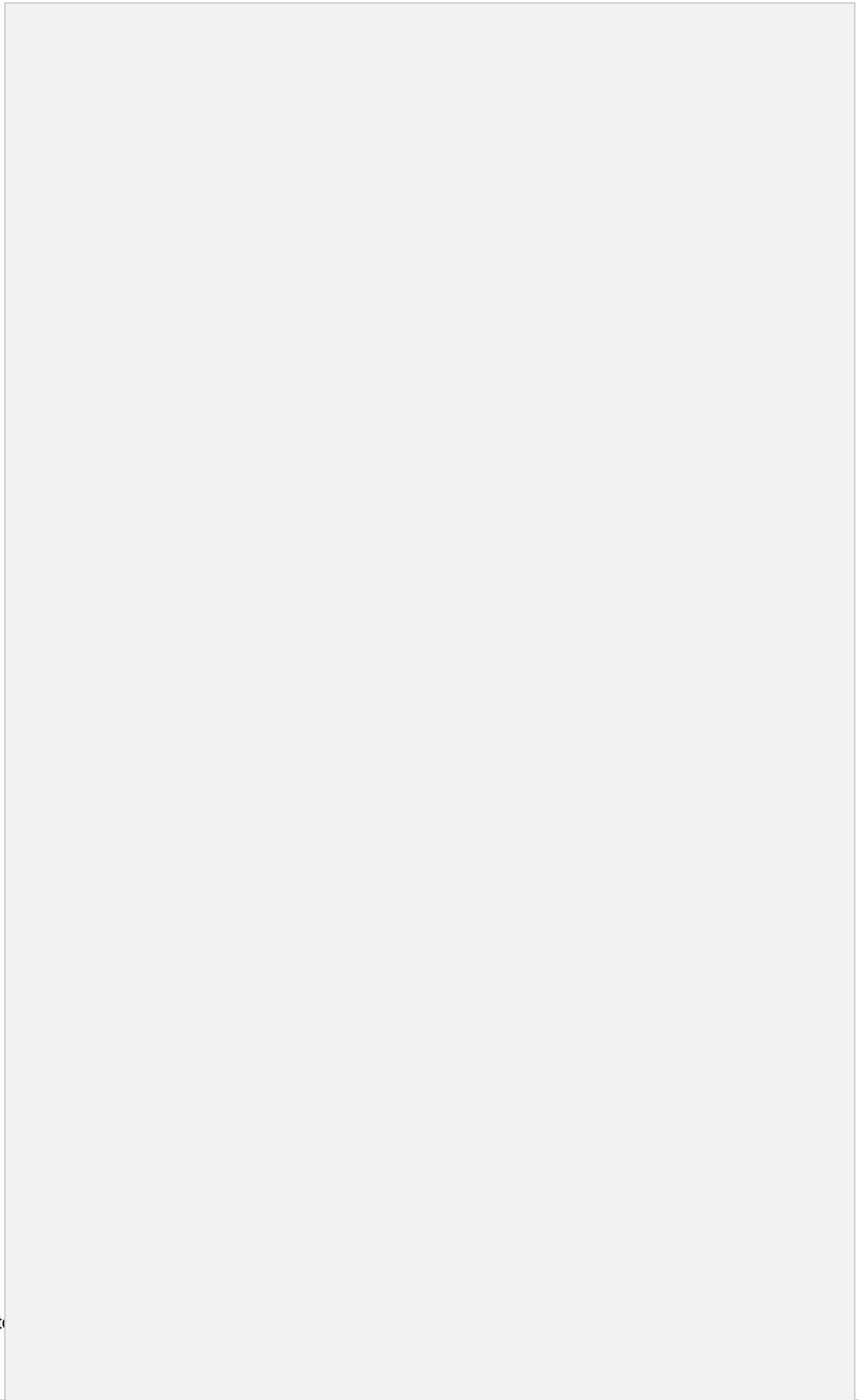


Figure 19. 390px customer portal, shown as a viewport crop. The full-height original is included in the evidence archive.

5. Integration and resilience findings

5.1 Database correctness and recovery

All 39 existing SQLite tests and all 38 PostgreSQL executions passed again. The PostgreSQL run mattered: behavior such as numeric NaN storage, transaction rollback and discount claims cannot be assumed equivalent just because SQLite is green.

The original-schema SQLite fixture upgraded without deleting the legacy user, contract or quotation. The migration versions were repeatable. Legacy quotations without reliable price snapshots still require re-quoting; the migration does not invent historical agreed prices.

The overlapping-billing tests used 500 due contracts and two workers. The fresh mixed-ledger run created exactly 500 invoices, with zero failures, in 690 ms locally. The PostgreSQL hardening log also passed its separate 500-contract overlap test; its batch took about 4.48 seconds in the shared test environment. Batch duration is not the same metric as the problem statement's two-second page-response requirement.

Failure-path tests checked that invalid pricing rolled back the invoice/cursor and did not auto-close away unbilled debt. Amendment revision checks and payment retry keys passed. These are the sorts of tests that make a financial application more trustworthy than a visually convincing prototype.

5.2 Mail delivery, failure and retry

The custom browser journey received three controlled SMTP messages: the invoice with PDF, a retried test notification and a legitimate customer's reset email. A separate security probe received the administrator reset message and reproduced the exposure while the notification status was **sent**, not merely logged.

An intentionally unreachable SMTP port produced a failed notification. Once the receiver was restored and the isolated row made due, two workers produced one receipt. The row recorded two attempts: one failed, one successful. Duplicate notification keys did not create another queued message.

This is durable, at-least-once behavior. A provider acknowledging late or a lost SMTP acknowledgment can still cause duplicate delivery. Stable Message-ID values help with recipient-side deduplication but do not create an exactly-once guarantee. No SPF, DKIM, DMARC, public certificate chain, mailbox spam placement or real provider credentials were validated here.

5.3 Production smoke and backup

The native production smoke served static frontend assets and API routes, exercised authentication and inspected security headers. Its SQLite backup was opened separately and passed an integrity check. The application no longer required the original external recording hooks for the tested production page.

Docker packaging and a GitHub Actions workflow are present, but neither a Docker-engine build nor a hosted workflow run was performed in this audit. PostgreSQL backup restoration, encrypted off-host storage, scheduler leadership across replicas, power-loss recovery and long-running worker endurance remain deployment acceptance work.

5.4 Dependencies and storage

The fresh dependency audit reported zero known vulnerabilities, including development dependencies. The clean audit did not identify the reset-link exposure because that defect is in first-party authorization/data projection, not a known package advisory.

Access and refresh tokens remain in localStorage. This is an inherited production-hardening concern, not a newly demonstrated XSS exploit. A session-design review, a broader XSS assessment, threat modelling and a real deployment review are still required. Do not describe the project as penetration-tested on the strength of this report.

6. Performance and original-layout checks

6.1 What was measured

The acceptance database contained 10,000 subscriptions, 60,000 invoices and 30,000 payments. Eight endpoints received 50 requests each at concurrency five. The browser performed five overview navigations/reloads and five transitions to each of ten workspace pages. Fetch quiescence and rendering frames were used rather than checking only for a changed heading.

The overview samples were **812, 383, 350, 349 and 351 ms**. The first includes initial sign-in, so the five values should not all be called cold-cache measurements. Every recorded page sample was below two seconds.

6.1 What was measured (tables)

Endpoint	Requests	Concurrency	p95 (ms)	Max (ms)
/subscriptions	50	5	17	22
/invoices	50	5	17	18
/invoices/outstanding	50	5	99	100
/payments	50	5	44	45
/dashboard/summary	50	5	151	151
/reports/aging	50	5	75	75
/analytics/revenue	50	5	326	326
/search?search=ACCEPT	50	5	72	72

Overview cold navigation/reload samples: 812, 383, 350, 349, 351 ms. Billing run: 500 invoices created from 500 due contracts, 0 failed, 690 ms with two overlapping workers; errors: 0.

Page / transition	Samples	p95 (ms)	Max (ms)
Subscriptions	5	198	198
Invoices	5	211	211
Payments	5	215	215
Customers	5	198	198
Products & plans	5	79	79
Quotations	5	199	199
Taxes & discounts	5	81	81
Reports	5	265	265
Audit log	5	198	198
Settings	5	65	65

Figure 20. Fresh local HTTP and browser timings from acceptance.json. These are bounded test observations, not an internet-facing SLA.

The slowest HTTP p95 was the revenue analytics endpoint at 326 ms. The slowest sampled workspace transition was Reports at 265 ms. With only five browser samples, the nearest-rank p95 is effectively the largest observed sample; it is not a statistically strong tail-latency estimate.

6.2 What the numbers do not establish

The test used local same-origin HTTP, an in-memory SQLite ledger and headless Chromium. It did not add WAN latency, mobile CPU throttling, a cold disk, hundreds of simultaneous users or production PostgreSQL network round trips. It did not establish a two-second guarantee for every possible page state.

There is also an important difference between fast reads and complete administration. The server could search a historical product in a 125-product catalog, but the advanced selector exposed only 100 editable records. A sub-second response is not a pass if the required record cannot be selected.

6.3 Visual preservation

The main stylesheet and server-record stylesheet retained their original hashes. Eight dashboard regions matched the original build's geometry and selected computed styles at 1440px and 390px; neither viewport had horizontal overflow in that comparison.

This is good evidence that the original visual identity was preserved. It does not prove every modal, empty state, dropdown and error state is pixel-identical, and it does not make the contrast/focus failures acceptable. The comparison intentionally blocked the old baseline's external recording/CDN hooks; it was a layout comparison, not approval of that baseline's external integrations.

7. Confirmed defect register

F-01 — Staff can obtain an administrator password-reset secret

Severity: Critical · Priority: P0 · Test: SEC-01 · Release gate: BLOCK

Expected: a staff account must not read another user's reset secret or become an administrator without administrator authorization.

Observed: the public forgot-password route queued a valid administrator reset URL. The staff-authorized notifications list returned its body. Redeeming that secret succeeded, and the replacement-password login returned ADMIN. The original administrator session was revoked as a consequence of the reset.

Isolated step	Result
Staff attempts the ordinary administrator-only staff-creation route	403 — direct permission check worked
Request reset for the seeded administrator email	200 — generic public response
Staff reads the notifications list	200 — message body included the reset URL
SMTP message status for that reset	sent — one controlled SMTP receipt
Redeem the exposed token with a replacement password	200 — password changed
Sign in with the replacement password	ADMIN returned
Reuse the original administrator access token	401 — original session revoked

The attack precondition is a staff session and knowledge of the administrator email. It does not require the administrator's old password or mailbox access. The test harness knew the original seeded password only to establish a baseline session and verify its revocation. It did not use that password to redeem the reset.

Root cause: the reset email body contains an actionable secret, while the generic notification listing returns whole notification records to staff. Relevant code is the forgot-password email composition in backend/auth.mjs, generic row projection in backend/app.mjs, and the staff-readable GET /api/notifications route. The flaw is server-side; hiding a panel in React would not fix it.

Required correction: return an explicit metadata allowlist from notification endpoints. Never expose reset URLs, message bodies or sensitive attachments through operational status reads. Separate security/account messages from ordinary invoice-delivery metadata. Review other listing/audit/export endpoints for the same projection pattern. In a real incident, invalidate exposed reset links and affected sessions and investigate access to persisted outbox data.

Retest acceptance: a staff account can inspect permitted delivery metadata but cannot recover the administrator reset token from any API response; a legitimate recipient can still reset through their mailbox; ownership, replay and expiry protections remain intact. This must become a permanent regression test, not a note beside a still-green suite.

Evidence: logs/security-probe.json, logs/security-probe.txt, and security_probe.mjs. Secrets are redacted. Figure 21 below illustrates the legitimate reset form, not the privilege-escalation result.

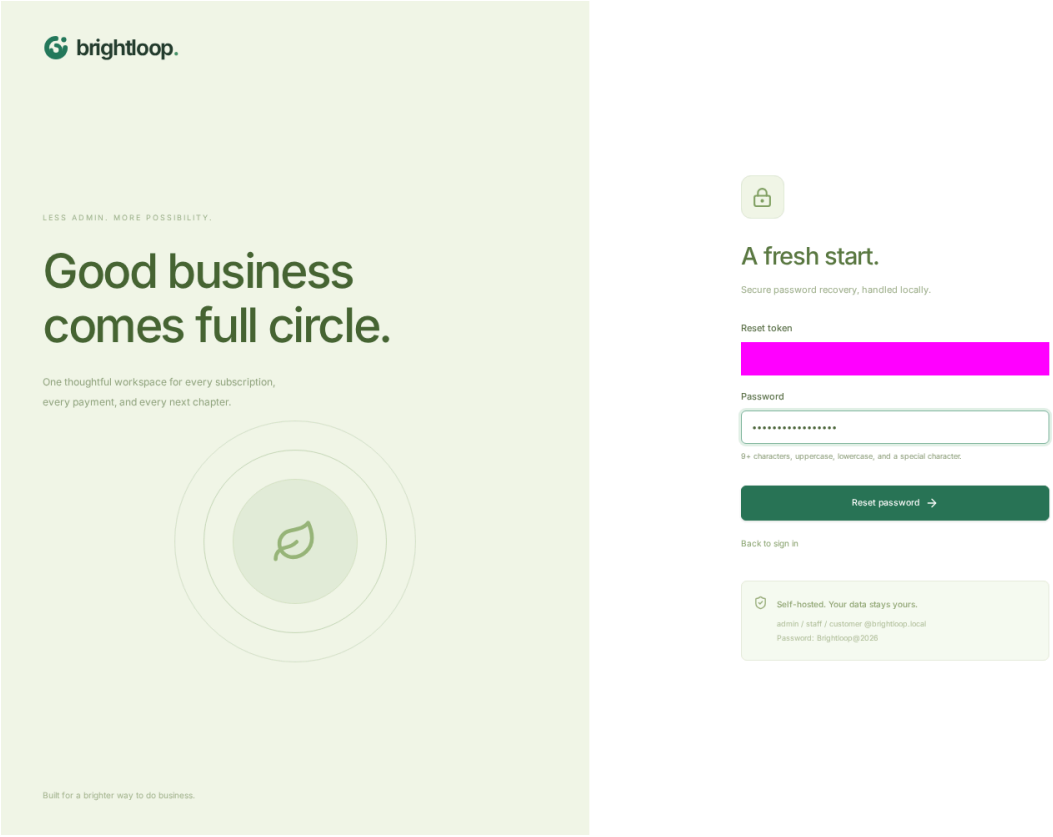


Figure 21. Legitimate browser password-reset flow from controlled SMTP. The token field is intentionally masked. The critical cross-role failure is proven by the redacted API/role evidence, not by this screenshot alone.

F-02 — Advanced catalog editing stops at the first 100 products

Severity: Medium · Priority: P1 · Test: SCALE-01

The isolated catalog contained 125 products. Advanced setup offered 100 existing records. “Historical audit product 119” was absent from that selector, although a server search returned exactly one matching record. The data was present; the editing route through the UI was incomplete.

The advanced editor builds its record choices from cached catalog data rather than a searchable server-backed record selector. The customer/plan pickers improved in the previous iteration do not solve this separate administration path.

Use a paginated/searchable selector that resolves the chosen record by ID. Do not merely increase a hard-coded limit. Retest products, plans, rules and quotation templates at realistic catalog sizes, including an older record outside the first page.

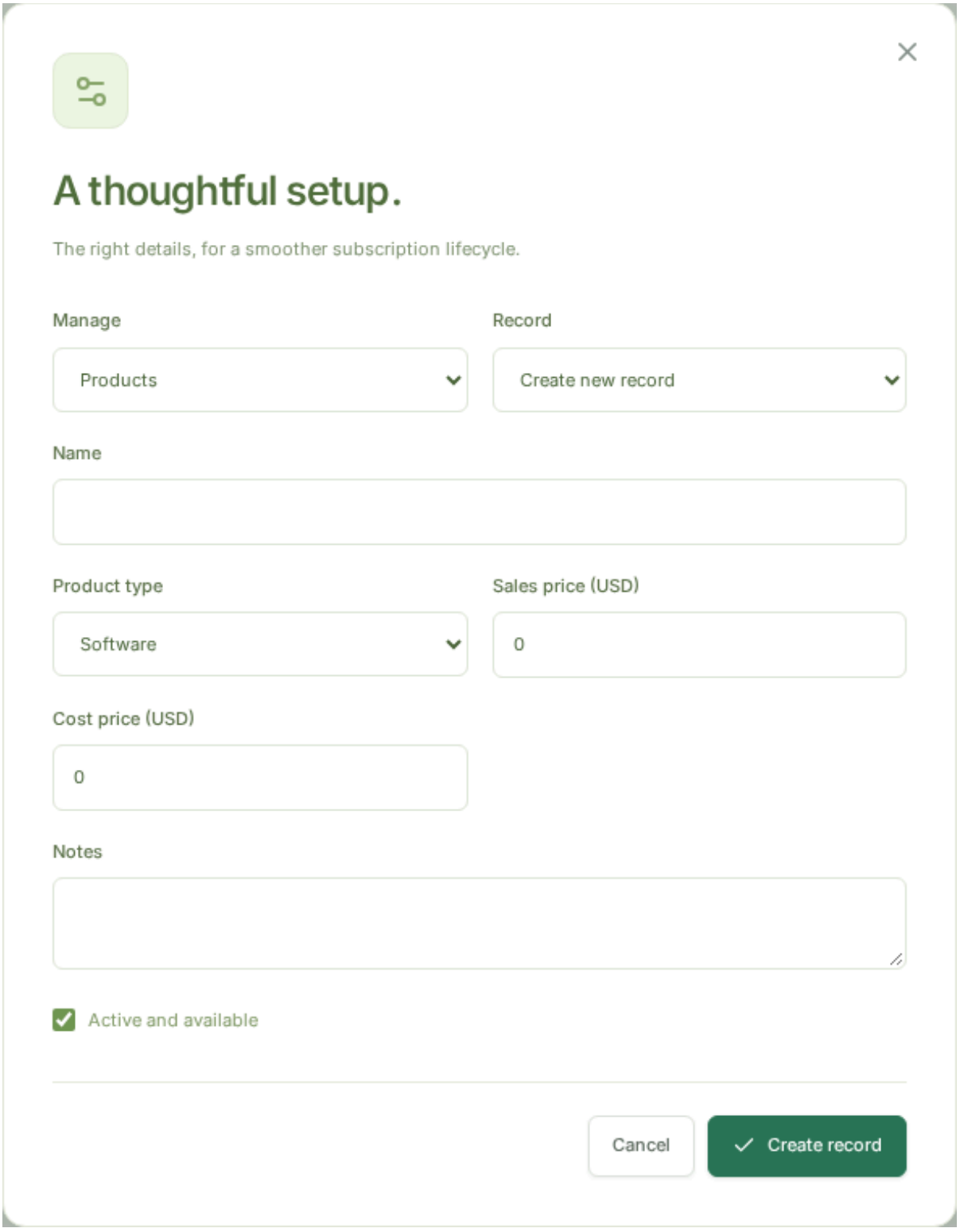


Figure 22. Actual advanced catalog editor during the 125-product fixture. The option count and absent target are recorded in browser-audit.json.

F-03 — Overview text contrast falls below the selected WCAG rules

Severity: Medium · Priority: P1 · Test: A11Y-01

Axe-core reported the serious color-contrast rule on 136 nodes in the tested admin overview. Targets and failure summaries are supplied in logs/accessibility.json. This was one scanned page under one theme and state; it is neither a whole-product defect count nor a full WCAG assessment.

Adjust the affected text, muted labels and backgrounds with the smallest practical changes. The visual-preservation constraint should be respected through restrained improvements, but should not be used to preserve unreadable contrast indefinitely. Retest every role, disabled state, badge and mobile view; add a screen-reader/manual review rather than relying solely on axe.

F-04 — Keyboard focus escapes the advanced-editor modal

Severity: Medium · Priority: P1 · Test: A11Y-02

After focusing a field in the open advanced editor and tabbing through its controls, focus escaped the dialog and reached BODY. The dialog declares itself modal, so the background should not participate in the active keyboard workflow.

Use a tested dialog/focus primitive or implement focus containment, focus restoration, accessible naming and appropriate background inertness. Confirm Shift+Tab, Escape, validation errors, nested overlays and restoration to the trigger. This finding is specifically verified for Advanced setup; other modals require the same review, not an assumption that all are broken in the same way.

F-05 — Settings misreport configured email delivery

Severity: Low · Priority: P2 · Test: UX-01

The process was configured for SMTP and had delivered actual messages, but Settings displayed “Console + outbox.log.” This makes it harder to distinguish a deliberately local preview from a real transport problem.

Expose safe runtime capability/status metadata and render the configured transport accurately. Do not expose credentials or raw provider errors. Review the scheduler label at the same time: a schedule string is not worker health. Verify local, SMTP, disabled and failure states.

7.1 Triage summary

Finding	Severity	Priority	Proposed owner	Closure evidence
F-01 reset-token exposure / role escalation	Critical	P0	Backend/authentication	New negative regression passes on both DB drivers; no sensitive projection
F-02 catalog record selection limit	Medium	P1	Frontend/catalog	Older records editable beyond page one in browser tests
F-03 text contrast	Medium	P1	UI/accessibility	Affected rules cleared; manual readability review
F-04 modal focus containment	Medium	P1	UI/accessibility	Keyboard loop and focus restoration verified
F-05 incorrect transport label	Low	P2	Frontend/operations	Runtime transport matches UI without leaking secrets

8. Requirement-by-requirement evaluation

The following matrix separates execution evidence from claims that remain limited. A covered row does not mean the entire feature is bug-free; F-01 is an example of a new combined path defeating individually passing checks.

Requirement area	What the fresh work shows	Status / limit
Accounts, roles, recovery	Real admin, staff and customer sessions; wrong password rejected; direct staff product/team writes 403; refresh replay rejected; logout revokes token; isolated recovery flow succeeds.	Implemented; confidentiality fails through the notifications path (F-01).
Products, variants, plans, rules	UI creation and DB read-back for product/cost/variant/plan/tax/discount/template; 100% validation checks passed; daily/weekly/monthly/yearly accepted; scoped rules applied.	Broadly covered; advanced selector stops at 100 products (F-02).
Quotations	Reusable template, quotation creation, one conversion, repeat conversion 409; existing expired/frozen-price/atomicity tests pass.	Covered for exercised states.
Lifecycle and amendments	Draft to Active, pause/resume, renew, illegal transition 409, plan flags, stale revision 409, future-dated customer amendment, issued invoice unchanged.	Covered; policies are intentional and need commercial explanation.
Recurring billing and money	2 seats, brand delta, two taxes, two discounts; $1,306 exact total; duplicate billing rerun no extra invoice; 500 due x 2 workers exactly 500; 39 SQLite and 38 PostgreSQL cases.	Strong in these scenarios; scheduler endurance and production PG load not tested.
Invoices, payments, PDF	Confirmed invoice, MIME PDF via SMTP, downloaded %PDF, partial $100 remains open, remaining $1,206 pays in full, invalid payments rejected, idempotency conflict 409.	Receipts only; no gateway, refund or chargeback processing.
Customer portal	Real customer ownership scoping; foreign record/PDF/edit/filter 403; safe catalog omits cost/notes; customer edits seats/add-on only; mobile has no horizontal overflow.	Covered; physical device and full assistive review not performed.
Search, filters, exports, audit	Server search finds a new contract and a customer beyond row 100; bounded/paginated reports; CSV/PDF captured; mutation audit rows exist.	Export breadth is paginated; spreadsheet file behavior not executed.
Operations and resilience	Native production smoke, SQLite backup integrity, SMTP failure and one retry under competing workers, notification dedupe, PostgreSQL schema migrations.	No Docker daemon run, hosted CI run or live provider verification.
Security	Auth, validation, concurrency and ownership checks pass; package audit shows zero known findings.	Critical first-party reset-token exposure remains; no full penetration test.
Performance and visual	10k/60k/30k mixed ledger; every recorded page sample under 2 s; CSS hashes and 8 regions match at 1440 and 390.	Local in-memory HTTP, 5-sample browser timings, no WAN or multi-user soak.
Accessibility and UX	Keyboard-driven creation/edit path, responsive 390px journey, sensible empty states and inline errors.	Contrast violation on 136 nodes, modal focus escape, misleading mail status label.
Documentation and packaging	README, policies, migrations, CI/Docker files and prior reports are present and consistent with tested behavior.	No hosted-CI status, production TLS review or independent code audit.

9. Revised score and release judgement

9.1 Same weights; new evidence

The earlier 93/100 was a self-assessment of the improved implementation. This audit keeps the same category weights but does not preserve the old result once contradictory evidence appears.

Category	Max	Earlier	Fresh	Evidence behind the fresh number
Accounts, roles and access	12	12	7	Direct boundaries pass; staff-to-admin reset path and secret projection fail.
Catalog, variants, plans, rules	13	13	12	Creation, validation and rule application pass; admin selection truncates at 100.
Subscription lifecycle and quotations	13	12	12	Lifecycle, conversion and prospective amendments hold in tested scenarios.
Billing, invoices and money	20	18	18	Exact arithmetic, duplicates, retry and payment state pass; no gateway processing exists.
Payments, PDF and delivery	9	9	9	Receipt state, PDF, MIME attachment and retry passed; provider/TLS not claimed.
Customer self-service	6	6	6	Ownership scoping, owned-field editing and mobile layout passed.
Finance, scale and performance	7	6	6	Sub-two-second local samples and bounded reads; no multi-user or WAN soak.
Architecture and maintainability	5	3	3	Real REST/data model and docs; monolithic frontend and deployment limitations remain.
Testing and correctness	6	6	5	Many suites pass, but the key cross-role reset regression was absent.
Security and operations	5	4	4	Hardening and backup pass; localStorage tokens and deployment proof remain.
UX, accessibility and docs	4	4	3	Strong visual preservation, but contrast, focus and transport-label defects remain.
Total	100	93	85	Security finding wins over an otherwise improved product.

The change from 93 to 85 is eight points: five in accounts/access for the reset-path confidentiality and effective staff/admin boundary; one in catalog completeness; one in testing for the missing critical negative regression; and one in UX/accessibility. These are evaluation judgements, not a formula that converts failed assertions into marks.

The critical finding is primarily charged to accounts/access rather than deducted again in security/operations. The existing security/operations deduction for localStorage sessions and unproven deployment review remains. Testing loses one point because the existing green gates do not catch the cross-role reset path; the new audit probe is supplied, but is not silently represented as an integrated production quality gate.

Functional score: 70/80. Engineering score: 15/20. Total: 85/100. This is not an external jury mark, production-readiness percentage or probability of winning a hackathon.

9.2 Release decision

Do not release with real accounts until F-01 is closed. A weighted score must not average away an administrator-takeover path. The finance workflows can be demonstrated on synthetic data, but that is different from approving a real subscription business deployment.

A sensible next sequence is:

1. Fix notification projections/security-message access; invalidate any exposed pending reset links where applicable.
2. Add the cross-role recovery regression to the normal suite and run it on SQLite and PostgreSQL.
3. Replace cached advanced-editor record selection with server-backed search.
4. Correct contrast, focus management and runtime status labels with restrained UI changes.
5. Rerun this audit, then a real deployment acceptance pass with HTTPS, the chosen database, provider SMTP and realistic device/network conditions.

No new score above 90 should be claimed merely by describing these changes. It should follow the retest evidence.

10. What remains untested or intentionally outside scope

Area	Honest limit
Physical Android/Termux execution	Narrow-screen Chromium was tested. Native dependency installation and full application execution on Termux were not.
Cross-browser behavior	Chromium only; no Safari/WebKit or Firefox run.
Production SMTP	Controlled SMTP socket only; no real provider/domain/TLS/mailbox acceptance test.
Payment processing	Receipt recording only; no gateway, bank movement, callback signing, refund or chargeback processing.
Hosted deployment	No Docker-engine build, hosted Actions run, CDN/reverse-proxy rollout or production TLS review.
PostgreSQL operations	Correctness/concurrency tested; live backup restoration, production data volume and WAN latency not tested.
Security	Targeted authorization/reset/validation checks, not a complete penetration test or exhaustive XSS review.
Accessibility	Automated overview scan and one modal keyboard probe; no full screen-reader or assistive-technology panel.
Scheduling	Service functions and an expiry fixture tested; no overnight cron endurance, failover or multi-replica leader rehearsal.
Accounting/compliance	Full anchored periods, UTC, one currency; no proration, statutory tax certification, revenue-recognition standard or full historical rule ledger.
Unbounded feature combinations	No finite automated run proves every permutation of rules, dates, catalog sizes and network failures.

The report is intentionally specific about these limits. Calling every integration “simulated successfully” would be misleading where there is no live integration to test.

Appendix A. New test register

The register below contains all 65 new named checks. It records the expected behavior and the actual result. Assertions inside a named check are not counted as separate tests. The security reproduction succeeds as an experiment but **fails the security expectation**; it is therefore marked FAIL.

ID	Check	Expected	Observed / status
UI-01	Login screen and invalid credentials	A wrong password is rejected; no workspace is opened	Expected result observed
UI-02	Administrator sign-in and overview	Admin signs in through the browser and sees real database aggregates	Expected result observed
UI-03	Overview CSV export	Downloaded CSV contains summary headings	Expected result observed
UI-04	Reporting period presets	Last month produces the previous calendar month	Expected result observed
UI-05	Create a product using Advanced setup	Name/type/sales/cost/notes persist in the database	Expected result observed
UI-06	Variant creation	Brand Odoo adds exactly 560 to the selected product	Expected result observed
UI-07	Create a billing plan	Monthly, minimum two seats, availability and all action flags persist	Expected result observed
UI-08	Tax validation and creation	A percentage above 100 is rejected; a scoped fixed tax is saved	Expected result observed
UI-09	Create a capped, product-specific discount	Fixed 20, two-seat minimum, dates and five-use cap persist	Expected result observed
UI-10	Reusable quotation template	Plan, two-seat product line and 14-day validity are saved	Expected result observed
UI-11	Create a quotation and convert it once	Selected template makes a frozen offer; Convert creates one linked subscription	Expected result observed
UI-12	Create subscription with variant and contract terms	Two seats at 100 + 560 each; Draft, expiry and Net 14 saved	Expected result observed
UI-13	Full lifecycle through the drawer	Draft → Quotation → Confirmed → Active, then pause/resume	Expected result observed
UI-14	Run billing and inspect calculated invoice	Subtotal 1320 + tax 136 − discounts 150 = 1306.00	Expected result observed
UI-15	Confirm invoice and send actual MIME attachment	Confirmed invoice stays at 1306; SMTP receiver gets a PDF attachment	Expected result observed
UI-16	Download invoice PDF	A real PDF is downloaded with the invoice number in its filename	Expected result observed
UI-17	Partial payment	Record 100 by bank; invoice remains Confirmed with a 1206 balance	Expected result observed
UI-18	Exact remaining payment	Record 1206 by UPI; invoice becomes Paid exactly once	Expected result observed
UI-19	Duplicate billing rerun	Running billing again creates no extra period for the tested subscription	Expected result observed
UI-20	Prospective staff amendment	Net 30 and future expiry save without changing the issued invoice	Expected result observed
UI-21	Global server search	Search finds the newly created subscription, not only the initial page	Expected result observed
UI-22	Finance reports and audit trail	Reports load; financial totals reconcile to source payment rows; mutations are audited	Expected result observed
UI-23	Admin staff creation	Only the admin UI exposes team creation and writes a STAFF account	Expected result observed
UI-24	Workspace settings	Display name persists locally; currency and timezone are visibly fixed	Expected result observed
UI-25	Customer login and role-specific navigation	Real CUSTOMER account sees its portal, not staff/admin pages	Expected result observed
UI-26	Customer-owned quantity and add-on editing	Customer can schedule three seats and Analytics Plus but cannot edit dates/terms	Expected result observed
UI-27	Mobile portal layout and navigation	390px layout has no horizontal overflow and the menu opens	Expected result observed
UX-02	Portal outstanding count matches collectible invoices	Card count equals server-filtered Confirmed outstanding invoices	Expected result observed
UI-28	Staff session and direct permission checks	Staff can operate subscriptions but direct catalog/team mutation is 403	Expected result observed
API-01	Portal isolation on detail, PDF and amendment	A foreign customer record cannot be fetched or changed	Expected result observed
API-02	Invalid dates and negative money	Malformed calendar date, negative price and 101% tax are rejected	Expected result observed
API-03	Product update and archive	An unused product can be updated and soft-archived with audit history	Expected result observed

API-04	SMTP failure and concurrent retry	Unreachable SMTP records failed, then two workers deliver one retry only	Expected result observed
UI-29	Sign-up and recovery through the browser	A new customer signs up and resets its password from a controlled mailbox link	Expected result observed
A11Y-01	Automated WCAG A/AA sweep of overview	No critical or serious violations	1 rule violations; 136 affected nodes; see accessibility.json
A11Y-02	Advanced editor keyboard focus containment	Tab stays inside the open modal	{"inside":false,"tag":"BODY","label":"brightloop.Audit WorkspaceYour workspaceWORKSPACEOverviewSub"}
SCALE-01	Advanced catalog selection beyond 100 records	An existing searchable product remains selectable for editing	{"databaseProducts":125,"editableOptions":100,"target":"Historical audit product 119","serverSearchFound":1,"selectorContainsTarget":false}
UX-01	Settings reflect configured mail transport	SMTP configuration is not labelled console/outbox-only	Storage mode Local API + database Billing scheduler Daily at 00:05 UTC Email delivery Console + outbox.log External services None required
UI-30	No unexpected browser runtime exceptions	No uncaught page errors during the journey	Expected result observed
M-01	Public health and unauthenticated isolation	Health 200; protected subscriptions 401	Expected result observed
M-02	Weak-password registration	A password missing the required character mix is 400	Expected result observed
M-03	Duplicate email and case-insensitive login	Mixed-case duplicate 409; mixed-case login 200	Expected result observed
M-04	Refresh rotation and replay	First refresh 200; reused refresh token 401	Expected result observed
M-05	Public reset response does not enumerate accounts	Known/unknown requests return identical messages	Expected result observed
M-06	Catalog input precision	Negative and three-decimal product prices return 400	Expected result observed
M-07	Variant create/read/update/delete	CRUD succeeds and modified adjustment reads back correctly	Expected result observed
M-08	All four plan frequencies	Daily, weekly, monthly and yearly are accepted and read back	Expected result observed
M-09	Plan edit and invalid availability	Price edit 200; reversed date range 400	Expected result observed
M-10	Minimum seats and plan availability	One seat is 400; not-yet-available plan is 409	Expected result observed
M-11	Draft cancellation and illegal transitions	Activate Draft is 409; cancel Draft yields Closed	Expected result observed
M-12	Plan action flags	Disabled pause, renew and early close each return 409	Expected result observed
M-13	Renewal extends the correct anniversary	Renew active subscription advances expiry by one monthly period	Expected result observed
M-14	Optimistic concurrency	A valid edit succeeds; stale revision returns 409	Expected result observed
M-15	Automatic closure on expiry	A deliberately clock-aged daily term closes at its expiry boundary	Expected result observed
M-16	Invoice cancellation	Draft invoice cancels; cancelled invoice cannot accept payment	Expected result observed
M-17	Payment guardrails	Future, overpaid, zero and sub-cent payments are rejected	Expected result observed
M-18	Payment idempotency	Same stable key returns the original payment, changed payload conflicts	Expected result observed
M-19	Tax and discount percentage/date validation	101 percent and reversed discount dates return 400	Expected result observed
M-20	Portal-safe catalog	Customer add-on catalog omits cost price and notes	Expected result observed
M-21	Date filters and pagination	Bounded report reads and page-size cap work; reversed range rejected	Expected result observed
M-22	Audit, billing history and notifications metadata	Authorized operational read endpoints return 200	Expected result observed
M-23	Notification deduplication	Repeated reminder runs do not duplicate already-queued keys	Expected result observed
M-24	Server customer scoping on list filters	Foreign customer filter is rejected instead of leaking rows	Expected result observed
M-25	Logout revokes access immediately	Previously issued access token is 401 after logout	Expected result observed
SEC-01	Staff cannot use an administrator reset link	Staff cannot read or redeem another role's password-reset secret	Staff notifications 200; administrator reset token exposed; reset 200; replacement-password login returned ADMIN; original admin token 401.

Appendix B. Existing regression inventory

The fresh quality gate executed the following 39 SQLite test cases. The first 38 core/hardening cases were also exercised on PostgreSQL; the legacy original-schema fixture is SQLite-only. Exact status and timing logs are provided separately. This inventory lists cases, not every assertion within each case.

#	Case executed in the fresh quality gate
1	regression: old active subscriptions and catalog records remain visible
2	regression: quotation price remains frozen across catalog edits
3	regression: confirmation preserves historical tax and discount snapshot
4	regression: pause/resume retains accrued unbilled periods
5	paused intervals skip only periods starting inside the pause and retain anchor
6	closing preserves previously accrued charges
7	regression: draft debt excluded and old confirmed debt reconciles with aging
8	regression: sub-cent payments rejected before creating zero-value entries
9	regression: full Draft to Quotation to Confirmed to Active sequence
10	billing anchors recover month-end and leap-year anniversaries
11	regression: historical revenue is independent of materialized summaries
12	versioned migration is repeatable and retains existing records
13	parallel confirmation never overspends final discount capacity
14	amendments preserve agreed prices, apply prospectively, and reject stale revisions
15	portal amendments enforce ownership, prevent contract overrides and keep cost private
16	global server search finds old records outside cached pages and scopes customers
17	session version revokes previously issued access tokens immediately
18	SMTP sends clickable resets and PDF attachments; failed delivery retries with one worker claim
19	payment retry keys prevent duplicate partial payments and reject changed payloads
20	add-on line is scheduled and can be removed without rewriting issued service
21	500 due subscriptions survive two overlapping billing workers without duplicate periods
22	failed pricing rolls back cursor/invoice and does not auto-close away unbilled debt
23	minimum seat quantity applies to the plan product, not an independent add-on
24	M1: server rejects weak password; unique emails are case-insensitive
25	M1: unauthenticated calls are 401; customer and staff cannot add staff
26	M1: refresh tokens rotate and cannot be replayed
27	M2/M3: admin catalog CRUD, variant delta, plan availability, staff mutation 403
28	M4: quote conversion is atomic, single-use, and rejects expired quotes
29	M5: illegal transitions return 409; pause and resume follow state machine
30	M7: expired and exhausted discounts are skipped; line tax precedes discount
31	M6: billing twice never duplicates an invoice; billing run scan is logged
32	M6/M7: confirmation consumes a capped discount exactly once
33	M8: partial payment stays Confirmed; exact balance transitions to Paid
34	M8: cancelled invoices, future payments, and overpayment are rejected
35	M9: server-side customer isolation covers lists, records, PDF and mutations
36	M6/M11: PDF is generated locally; notifications are idempotent
37	M10/M11: paginated date-bound reports, summaries and mutation audits
38	M1: reset tokens are expiring, single-use and password policy is enforced
39	genuine original schema upgrades without deleting old business data

Appendix C. Evidence and reproducibility

C.1 Evidence package

- logs/audit-summary.json: final counts, revised assessment and release decision.
- logs/check-register.json: all new checks, their expectations and observed outcomes.
- logs/browser-audit.json: browser/API journey, screenshot descriptions, sanitized request paths/statuses and observations.
- logs/api-matrix.json: additional validation and lifecycle edge cases.
- logs/security-probe.json: redacted privilege-escalation result on isolated data.
- logs/accessibility.json: axe rule, target selectors and failure summaries.
- logs/acceptance.json: fresh mixed-ledger HTTP/browser timings and overlap counts.
- logs/quality-gate.txt, coverage.txt, postgres.txt, demo-ui.txt, api-ui.txt, production.txt, visual-fresh.txt, npm-audit.json: raw execution evidence.
- logs/provenance.json: audited commit, source hashes and clean working-tree record.
- screenshots/: full original application PNGs; report_figures/: clearly identified print crops.
- downloads/: the actual invoice PDF and overview CSV captured by Playwright.
- Audit harnesses: browser journey, API matrix, targeted security probe and acceptance harness.

The evidence archive deliberately excludes runtime databases, .env files, live credentials, node_modules, local outbox bodies and the harness-debug failure captures taken while selectors were being corrected. The synthetic reset-token values are redacted from the shared security result. Script fixture passwords are test-only, not deployment credentials.

C.2 Reproducing the checks

Install the repository with Node 24 and npm ci. The existing suites use the commands below. PostgreSQL requires an isolated test service; do not point test tools at a production database.

```
npm run check
npm run test:coverage
npm run test:ui
npm run test:local-ui
TEST_DATABASE_URL=<isolated-postgresql-url> npm run test:postgres
npm run test:production
npm run test:visual
```

The visual suite additionally needs the built original baseline checkout described in the repository documentation. The supplied audit harnesses use explicit workspace paths; adapt ROOT/OUT, rebuild the audit frontend with VITE_API_URL=/api and install axe-core before running them elsewhere. They are reproducibility material, not a one-click production deployment.

The security and exploratory runners intentionally return a non-zero status while their failing expectations remain. Do not change that into a green exit merely to make a pipeline pass. Confirm the intended code correction and rerun the checks instead.

C.3 Final note

The project now has a believable billing story, not just attractive screens. Its pricing snapshots, payment arithmetic and retry behavior held up well in the scenarios exercised. The important next step is equally concrete: close the reset-token exposure and the smaller completeness/accessibility gaps, then earn the higher score again with a retest. That is a stronger submission story than an unqualified claim that everything is finished.